

Step 3 — A Triage Agent That Extracts a Structured Ticket From Free Text — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — Step 3: the chat can't file tickets yet. This step adds the first real agent — Triage — which turns a complaint into a checked `SupportTicket`. For now you trigger it by hand with `/extract`, so you can test it on its own before Step 4 automates the choice. This is also where the app learns to survive every common failure. **If not:** a support desk that can't produce structured tickets is just a chatbot, and one rate limit or bad reply would crash the whole session.

After this step the project runs as a chat app that can also file tickets. Files from Steps 1-2 that aren't shown here don't change.

Basic Version

Approach 1 — the ticket, and two of the failures

Story — `schemas.py` (Basic): the ticket's exact shape, as a Pydantic model. **If not:** you'd be hand-parsing the model's text and hoping for the right keys.

```
# src/supportdesk/schemas.py
from typing import Literal

from pydantic import BaseModel

class SupportTicket(BaseModel):
    customer_name: str | None
    issue_category: Literal["billing", "technical", "account_access", "other"]
    urgency: Literal["low", "medium", "high"]
    summary: str
```

Story — `agents/triage.py` (Basic): one structured-output call, the way Doc04 did it, with its own system prompt. (Also create an empty `src/supportdesk/agents/__init__.py`.) **If not:** there'd be no agent to call.

```
# src/supportdesk/agents/triage.py
from openai import OpenAI
from pydantic import ValidationError

from supportdesk.models.llm import MODEL
from supportdesk.schemas import SupportTicket

def triage_agent(client: OpenAI, message: str) -> SupportTicket | None:
    messages = [
        {"role": "system", "content": "Extract a support ticket."},
        {"role": "user", "content": message},
    ]
    try:
        completion = client.beta.chat.completions.parse(
            model=MODEL, messages=messages, response_format=SupportTicket
        )
    except ValidationError:
        return None
    return completion.choices[0].message.parsed
```

Story — `main.py` (Basic): Step 2's loop plus an `/extract` branch and two `except` blocks. **If not:** the triage agent couldn't be reached from the app.

```
# src/supportdesk/main.py
import sys

import openai

from supportdesk.agents.triage import triage_agent
from supportdesk.config import load_config
from supportdesk.exceptions import MissingConfigError
from supportdesk.models.llm import (
    check_api_key_at_startup,
    create_client,
    stream_message,
)
from supportdesk.utils.history import trim_history
from supportdesk.utils.prompts import load_prompt

MAX_HISTORY_TOKENS = 3000

def main() -> None:
    try:
        config = load_config()
```

```

except MissingConfigError as e:
    print(f"Setup problem: {e}")
    sys.exit(1)

client = create_client(config)
if not check_api_key_at_startup(client):
    print("Login failed – check OPENAI_API_KEY in your .env file.")
    sys.exit(1)

history = [{"role": "system", "content": load_prompt("concierge")}]
while True:
    message = input("You: ").strip()
    if message == "/quit":
        break
    try:
        if message.startswith("/extract "):
            text = message.removeprefix("/extract ")
            print(triage_agent(client, text))
        else:
            trim_history(history, MAX_HISTORY_TOKENS)
            print("Assistant: ", end="", flush=True)
            stream_message(client, history, message)
    except openai.AuthenticationError:
        print("Your API key stopped working.")
        sys.exit(1)
    except openai.RateLimitError:
        print("Rate limited – try again in a moment.")

if __name__ == "__main__":
    main()

```

This works on the 3 sample inputs. But the prompt doesn't forbid inventing a name, so input 2 can come back with a made-up one. A refusal isn't checked. A too-long message still crashes the app. And nothing is logged or tested.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a strict agent, every failure handled, sample data and tests

Story — `schemas.py` : the contract between the model and your code. `str | None` lets "no name given" be a real answer, and `Literal` limits category and urgency to exact values. **If not:** a required `customer_name` would force the model to invent one, and a free-text category could come back as "Billing issue" — valid text that breaks any `if category == "billing"`.

```
# src/supportdesk/schemas.py
from typing import Literal

from pydantic import BaseModel

class SupportTicket(BaseModel):
    # why: str | None, not str – a required name would force the model
    # to invent one when the customer never gave it.
    customer_name: str | None
    # how: Literal means the model can only pick one of these exact values
    issue_category: Literal["billing", "technical", "account_access", "other"]
    urgency: Literal["low", "medium", "high"]
    summary: str
```

Story — `prompts/triage.txt` : a strict prompt for a strict job — it says outright never to guess a name, and when urgency is "high". **If not:** the schema allows `None`, but nothing *tells* the model to use it, so it may still fill in a name.

`src/supportdesk/prompts/triage.txt`

```
You are a support triage agent. Turn the customer's message into a
support ticket. Fill customer_name only if the customer states their
name - otherwise leave it null, never guess one. Use urgency 'high'
only if the customer says it is urgent or needed today.
```

Story — `agents/triage.py` : the Triage agent — its own prompt, no chat history, one structured call. It checks `refusal` before reading `parsed`, and returns `None` on any unusable answer, so the caller has exactly two cases to handle. **If not:** a refusal would come back as `parsed=None` with no explanation, and invalid output could reach code that trusts it.

```
# src/supportdesk/agents/triage.py
from openai import OpenAI
from pydantic import ValidationError

from supportdesk.models.llm import MODEL
from supportdesk.schemas import SupportTicket
from supportdesk.utils.logger import get_logger
from supportdesk.utils.prompts import load_prompt

def triage_agent(client: OpenAI, message: str) -> SupportTicket | None:
    # when: the logger is fetched here, not at import time, so LOG_LEVEL
    # from .env is already loaded by load_config() when it is set up
    logger = get_logger(__name__)
    # why: no chat history – every complaint is judged on its own
    messages = [
        {"role": "system", "content": load_prompt("triage")},
        {"role": "user", "content": message},
    ]
    try:
        completion = client.beta.chat.completions.parse(
            model=MODEL, messages=messages, response_format=SupportTicket
        )
    except ValidationError as e:
        # why: bad structured output is reported, never passed on
        logger.warning("Triage output did not match SupportTicket: %s", e)
        return None

    reply = completion.choices[0].message
    # how: check refusal before parsed – the model can decline instead
    if reply.refusal:
        logger.warning("Triage agent refused: %s", reply.refusal)
        return None
    return reply.parsed
```

Also create an empty `src/supportdesk/agents/__init__.py`.

Changed from Step 2: the startup and the chat branch are the same. What's new: 3 imports, `print_ticket()`, the `/extract` branch, and the `try / except` blocks around the loop body.

Story — `main.py`: Step 2's loop plus `/extract`, a readable `print_ticket()`, and one `except` per failure. Each failure gets the reaction Doc04's error table gives it: exit on a bad key, ask the user to wait on a rate limit, and drop a too-long message. (The 4th failure, bad structured output, is already handled inside `triage_agent()`.) **If not:** any one of these would end the session with a traceback, and the user would lose the whole conversation.

```

# src/supportdesk/main.py
import sys

# new in Step 3: openai (for its error classes), triage_agent, SupportTicket
import openai

from supportdesk.agents.triage import triage_agent
from supportdesk.config import load_config
from supportdesk.exceptions import MissingConfigError
from supportdesk.models.llm import (
    check_api_key_at_startup,
    create_client,
    stream_message,
)
from supportdesk.schemas import SupportTicket
from supportdesk.utils.history import trim_history
from supportdesk.utils.logger import get_logger
from supportdesk.utils.prompts import load_prompt

# why: far below gpt-4o-mini's real limit (128,000 tokens), so the rough
# 4-characters-per-token guess can be off and still be safe
MAX_HISTORY_TOKENS = 3000

# new in Step 3
def print_ticket(ticket: SupportTicket) -> None:
    name = ticket.customer_name
    if name is None:
        # why: show clearly that no name was given – never a made-up one
        name = "(not given)"
    print("Ticket created:")
    print(f"  customer: {name}")
    print(f"  category: {ticket.issue_category}")
    print(f"  urgency: {ticket.urgency}")
    print(f"  summary: {ticket.summary}")

def main() -> None:
    # why: config first – a missing OPENAI_API_KEY stops the app here,
    # with one clear line, before anything else runs
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)

```

```

client = create_client(config)
if not check_api_key_at_startup(client):
    logger.error("Authentication failed – check OPENAI_API_KEY.")
    print("Login failed – check OPENAI_API_KEY in your .env file.")
    sys.exit(1)

history = [{"role": "system", "content": load_prompt("concierge")}]
print("SupportDesk AI – type a message, or /quit to exit.")
print("Type /extract <complaint> to file a ticket.")

while True:
    message = input("You: ").strip()
    if message == "/quit":
        break
    if message == "":
        continue

    # new in Step 3: try / except around the loop body, and /extract
    try:
        if message.startswith("/extract "):
            # when: a temporary manual switch – Step 4's router
            # replaces it with an automatic decision
            text = message.removeprefix("/extract ")
            ticket = triage_agent(client, text)
            if ticket is None:
                print("Sorry, I couldn't turn that into a ticket.")
            else:
                print_ticket(ticket)
        else:
            trim_history(history, MAX_HISTORY_TOKENS)
            print("Assistant: ", end="", flush=True)
            stream_message(client, history, message)
    except openai.AuthenticationError:
        # how: a bad key is permanent – no retry fixes it, so exit
        logger.error("API key stopped working mid-session.")
        print("Login failed – check OPENAI_API_KEY in your .env file.")
        sys.exit(1)
    except openai.RateLimitError:
        # how: the client already retried twice before this reached us
        logger.warning("Rate limited after the client's retries.")
        print("We're busy right now – wait a moment and try again.")
    except openai.BadRequestError as e:
        logger.warning("Request rejected (likely too long): %s", e)
        # why: the message that broke the limit is still at the end of
        # history – remove it, or every next turn resends it and fails
        if history[-1]["role"] == "user":
            history.pop()

```

```

        print("That message was too long to handle – please shorten it.")

if __name__ == "__main__":
    main()

```

Why these `except` blocks sit in `main.py`: this is the only place that knows how to talk to the *user* (print a message, or exit). The agents just raise or return `None`.

Story — `data/sample/messages.json`: the 3 test complaints from the README, stored once. **If not:** the script and the test would each keep their own copy, and the copies would drift apart.

`data/sample/messages.json`

```

{
  "triage": [
    "Hi, I'm Sarah Khan. I was charged twice for my subscription this month and I need it fixed today.",
    "can't log into my account, tried resetting password twice, still nothing",
    "just wondering what your refund policy is, no rush"
  ]
}

```

(The first message is wrapped onto 2 lines here only to fit the page — in your file keep it on **one** line, because a JSON string can't contain a line break.)

Story — `scripts/triage_samples.py`: runs Triage on every sample and prints each ticket, for you to judge — for example after rewording `triage.txt`. **If not:** checking a prompt change would mean typing 3 complaints by hand each time.

```

# scripts/triage_samples.py
# Runs the Triage agent on every sample complaint and prints each ticket.
# Run from the repo root: python scripts/triage_samples.py
# Makes real API calls – needs a valid OPENAI_API_KEY in .env.
import json

from supportdesk.agents.triage import triage_agent
from supportdesk.config import load_config
from supportdesk.models.llm import create_client

SAMPLES_PATH = "data/sample/messages.json"

def main() -> None:
    # why: a script, not a test – it prints tickets for a person to read
    # and judge, e.g. to compare the effect of a change to triage.txt

```



```

config = load_config()
client = create_client(config)
with open(SAMPLES_PATH) as samples_file:
    # how: read the file as text, then json.loads() it (Doc02)
    samples = json.loads(samples_file.read())

for message in samples["triage"]:
    print(f"\nMessage: {message}")
    ticket = triage_agent(client, message)
    if ticket is None:
        print("No ticket – the Triage agent couldn't extract one.")
    else:
        print(f"Ticket: {ticket}")

if __name__ == "__main__":
    main()

```

Story — `tests/unit/test_schemas.py` : checks the schema allows a missing name and rejects an unknown urgency — free and instant. **If not:** someone could change `customer_name` to a plain `str` and nothing would warn them until the model started inventing names.

```

# tests/unit/test_schemas.py
# Checks the Pydantic schemas accept good data and reject bad data.
# No API calls, no API key needed.
# Run from the repo root: python tests/unit/test_schemas.py
from pydantic import ValidationError

from supportdesk.schemas import SupportTicket

def test_ticket_without_a_name() -> None:
    # a missing name must be allowed – the model should never invent one
    ticket = SupportTicket(
        customer_name=None,
        issue_category="account_access",
        urgency="medium",
        summary="Customer can't log in after two password resets.",
    )
    print(f"case 1 (no name allowed)      -> {ticket.customer_name}")

def test_ticket_rejects_unknown_urgency() -> None:
    try:
        SupportTicket(
            customer_name="Sarah Khan",

```

```

        issue_category="billing",
        urgency="urgent",
        summary="Charged twice.",
    )
except ValidationError:
    print("case 2 (bad urgency)          -> ValidationError")

if __name__ == "__main__":
    test_ticket_without_a_name()
    test_ticket_rejects_unknown_urgency()

```

Story — `tests/integration/test_live_agents.py` : the checks that need the real model — memory, the startup key check, and the 3 sample tickets, including "no invented name". It's the same style as Doc04's `test_chat_client.py` . **If not:** "it works" would only mean "it worked the one time I tried it by hand".

```

# tests/integration/test_live_agents.py
# Runs the project's checks against the real OpenAI API.
# Run from the repo root: python tests/integration/test_live_agents.py
# Makes real API calls – needs a valid OPENAI_API_KEY in .env.
import json

from openai import OpenAI

from supportdesk.agents.triage import triage_agent
from supportdesk.config import load_config
from supportdesk.models.llm import (
    check_api_key_at_startup,
    create_client,
    send_message,
)

SAMPLES_PATH = "data/sample/messages.json"

def load_samples() -> dict:
    with open(SAMPLES_PATH) as samples_file:
        # how: read the file as text, then json.loads() it (Doc02)
        return json.loads(samples_file.read())

def test_memory(client: OpenAI) -> None:
    # case 1: turn 3 must be able to use what was said in turn 1
    history = [{"role": "system", "content": "You are a helpful assistant."}]
    send_message(client, history, "My order number is 4471.")

```

```

send_message(client, history, "Thanks, that's all for now.")
reply = send_message(client, history, "What was my order number?")
print(f"case 1 (memory)          -> {reply}")

def test_bad_key() -> None:
    # case 2: a wrong key is caught at startup, not mid-chat
    bad_client = OpenAI(api_key="sk-fake-key-123")
    key_ok = check_api_key_at_startup(bad_client)
    print(f"case 2 (bad key ok?)   -> {key_ok}")

def test_triage(client: OpenAI, samples: dict) -> None:
    # case 3: each sample complaint becomes a ticket
    for message in samples["trriage"]:
        ticket = triage_agent(client, message)
        print(f"case 3 (trriage)      -> {ticket}")

    # case 4: the second complaint gives no name – none may be invented
    second = triage_agent(client, samples["trriage"][1])
    if second is not None and second.customer_name is None:
        print("case 4 (no name)        -> OK, customer_name is None")
    else:
        print("case 4 (no name)        -> WRONG, a name was invented")

if __name__ == "__main__":
    config = load_config()
    client = create_client(config)
    samples = load_samples()
    test_memory(client)
    test_bad_key()
    test_triage(client, samples)

```

Run everything:

```

python tests/unit/test_history.py
python tests/unit/test_schemas.py
python tests/integration/test_live_agents.py
python scripts/triage_samples.py
python -m supportdesk.main

```

Expected output of the unit test:

```

case 1 (no name allowed)      -> None

```

```
case 2 (bad urgency)          -> ValidationError
```

Expected output of the integration test (tickets shown shortened — the model's wording varies):

```
case 1 (memory)              -> Your order number was 4471.
case 2 (bad key ok?)         -> False
case 3 (triage)              -> customer_name='Sarah Khan' issue_category='billing'
                               urgency='high' summary='...'
case 3 (triage)              -> customer_name=None issue_category='account_access'
                               urgency='medium' summary='...'
case 3 (triage)              -> customer_name=None issue_category='other'
                               urgency='low' summary='...'
case 4 (no name)             -> OK, customer_name is None
```

(Each ticket is really one line; it's wrapped here to fit the page.)

In the app, `/extract` can't log into my account, tried resetting password twice prints a ticket with `customer: (not given)`.

Difference from Basic: a strict prompt file that forbids guessing a name; a refusal check; logging; a readable ticket; all 4 failures handled with the right reaction for each; and sample data, a script and tests, so "it works" is something you can re-run, not just remember.

Which one should you actually write? Intermediate. The checklist's "survives every failure" and "tested on all 3 inputs, including the missing name" both need it, and Step 4 reuses `triage_agent()` and the whole `except` block unchanged.